your quantum project from zero to presentation

examace - beginner guide, worked examples & actual code

start with pages 1-14 for the 30-minute reading route. the appendices contain every line of the shared records, calculation, solver, export, replay & checks, with the reason for important lines explained immediately before them.



one saved profile - both solvers receive the same scores, weights & budget

what you built

a student answers six calculus questions. the app is intended to turn those answers into three initial topic scores. your optimizer chooses study time. a separate experiment asks whether a small simulated qaoa circuit can find a good answer to the same allocation problem.

what the result actually says

the exact solver found utility 1.416667. the sampled qaoa plan had utility 0.666667, a gap of 0.75. 86 of 1024 sampled measurements were feasible. the optimizer stopped at its 30-evaluation cap. this is a valid experiment with an imperfect result.

how to read

minutes 0-10 - pages 2-6, understand the decisions & energy. minutes 10-20 - pages 7-11, understand amplitudes, gates & qaoa. minutes 20-30 - pages 12-14, trace the run, rehearse the explanation & answer the questions. use the appendix when you want the exact code.

source snapshot - 545b148. the recorded r4 simulation used source commit 2a74c42 with a clean working tree. later commits add evidence & checks.

02 - the ordinary python problem

before quantum computing, decide what a good answer means. there are three topics. each may receive 0, 30 or 60 minutes. a dictionary such as {"T1": 0, "T2": 60, "T3": 30} is a candidate answer.

scores & weights

proficiency is the initial score from two diagnostic questions, so real diagnostic scores are 0, 0.5 or 1. a weight is a heuristic importance number between 0.5 & 1. the reviewed assessment counts determine it. the optimizer consumes these values; it does not grade answers.

```
priority = weight * (1 - proficiency)
```

a topic with weight 1 & proficiency 0.2 has priority 0.8. raising proficiency reduces its remaining priority. this is a project rule, not evidence that 30 minutes will produce a particular grade improvement.

diminishing returns

minutes	utility from a topic with priority 0.8
0	0
30	0.8
60	0.8 + 0.4 = 1.2

the second block earns half the first block value. utility means value under this chosen formula. it is not measured learning.

why exact is easy here

there are 3 choices for each of 3 topics, giving 3*3*3 = 27 allocations. checking all 27 finds the exact optimum under the supplied model. ties prefer less study time, then more blocks for T1, T2, T3 in that order. it is fine to leave time unused when studying adds zero utility.

03 - a hand-calculated answer

use test-only weights of 1 for every topic, scores T1=.8, T2=.2, T3=.4 & a 90-minute budget. these scores are an optimizer test, not all obtainable from the six-item diagnostic.

topic	first block / second block
t1	0.2 / 0.1
t2	0.8 / 0.4
t3	0.6 / 0.3

you can afford three blocks. selecting T2 first, T3 first & T2 second gives 0.8+0.6+0.4 = 1.8. the allocation is T1=0, T2=60, T3=30. selecting a second block requires its first block.

the actual fictional run uses different weights

case-a scores are T1=1, T2=0, T3=.5. actual weights are T1=5/6, T2=2/3, T3=5/6. priorities are therefore 0, 2/3, 5/12. the best allocation is still 0/60/30, but utility is 2/3+1/3+5/12 = 17/12, approximately 1.416667.

why this distinction matters

copying 1.8 or 2.0 into the report would compare the wrong problem. both solvers must calculate values using the exported snapshot. the report retains the entire snapshot so someone else can check what was actually solved.

```
request.model_copy(update={"requested_solver": "qaoa"})
```

the copy changes only the solver choice. scores & budget remain identical. topic weights come from the same snapshot for both solves.

04 - bits are yes/no decisions

a classical bit is a value restricted to 0 or 1. here it means whether a study block is selected. nothing quantum is needed yet.

first / second	meaning
0 / 0	neither selected - 0 minutes
1 / 0	first selected - 30 minutes
1 / 1	both selected - 60 minutes
0 / 1	second without first - invalid

```
bits = [0, 0, 1, 1, 1, 0, 0, 0, 0]
```

positions 0 & 1 are T1 first/second. positions 2 & 3 are T2. positions 4 & 5 are T3. the last three positions represent unused blocks. this example selects 0/60/30 & leaves zero whole blocks unused.

why nine bits rather than six

budget_blocks = budget_minutes//30. we express the budget as selected blocks + unused blocks = budget_blocks. the unused count is called slack. three extra bits represent it as y0 + 2*y1 + 4*y2, from 0 through 7. budgets allow at most 6 study blocks, so this is sufficient.

```
empty_90_min_plan = [0,0,0,0,0,0,1,1,0]
```

the last bits encode 1+2=3 unused blocks. zero selected + three unused = three available. a 95-minute budget still has three blocks; its extra five minutes remain unused separately.

nine binary positions give 512 possible strings. many are invalid. that is different from the 27 legal topic allocations checked by the exact solver.

05 - coefficient is just a multiplier

```
3*x
```

3 is the coefficient. a single-bit coefficient multiplies one bit. a pair coefficient multiplies the product of two bits. these are ordinary algebra terms.

x / y	x*y
0 / 0	0
0 / 1	0
1 / 0	0
1 / 1	1

a pair term activates when both bits are 1

```
energy = -0.8*x - 0.4*y
```

x selects the first block, y selects the second. selecting them reduces energy by their utility. negative utility turns a maximization problem into a minimization problem.

catch an illegal second block

```
penalty = 3.4*y*(1-x)
# expand brackets
penalty = 3.4*y - 3.4*x*y
```

if y=1 & x=0, penalty is 3.4. if both are 1, +3.4 & -3.4 cancel. if y=0, both terms are zero. this creates a single-bit contribution +3.4 for y & a pair contribution -3.4 for x*y.

including utility gives -0.8*x +3.0*y -3.4*x*y. with x=0,y=1 it is 3.0, which is unfavorable. with both selected it is -1.2, the negative of the valid 60-minute utility. budget penalties are additional.

06 - the complete energy formula

```
P = 1 + 1.5*sum(priorities)

E = -utility + P*(selected + slack - B)**2

E += P*sum(second*(1-first))
```

B is the number of affordable blocks. the squared budget error is zero exactly when selected+slack=B. any nonzero integer error squares to at least one. the ordering term is zero unless a second block appears without its first.

the maximum possible utility is 1.5*sum(priorities). P is one greater than that. therefore even one violation costs more than all possible utility can save. a feasible empty plan always exists by putting the whole block budget into slack.

from brackets to coefficients

write selected+slack as sum(c_j*z_j), where c=(1,1,1,1,1,1,1,2,4). expand its square. because each bit z is 0 or 1, z*z equals z. diagonal square terms therefore become single-bit terms. cross terms become bit-pair terms.

```
C = P*B**2

L[j] = P*c[j]**2 - 2*P*B*c[j] - value[j]

Q[j,k] = 2*P*c[j]*c[k]
```

add +P to a second-block linear term & -P to its same-topic pair term. those are the expanded ordering penalties from the previous page. store each pair once using j<k.

why keep the constant

C does not change which answer is best, but removing it changes numerical energies. keeping it lets tests show the direct formula & the expanded polynomial agree.

07 - qubits, amplitudes & measurement

a qubit is a quantum system with two measurement outcomes in the basis used here, 0 & 1. before measurement, we describe its state using two amplitudes. an amplitude is a number used to calculate outcome probabilities.

```
state = a*|0> + b*|1>

probability(0) = abs(a)**2

probability(1) = abs(b)**2
```

the vertical-bar notation is just a label for a basis state. |0> & |1> name the two outcomes. the probabilities must sum to 1.

numbers can have signs & imaginary parts

the imaginary unit i satisfies i*i=-1. an amplitude u+v*i has squared magnitude u*u+v*v. for example .5+.5*i has probability .25+.25=.5. you do not need to implement complex arithmetic yourself; numpy & qiskit handle it.

a=b=1/sqrt(2) gives half probability for each outcome. a=1/sqrt(2), b=-1/sqrt(2) gives the same measurement probabilities now, but those states can behave differently under a later operation.

nine qubits

the simulator stores 512 amplitudes, one for every nine-bit basis string. their squared magnitudes sum to 1. one measurement gives one string, not a readable list of all 512 answers. the full amplitude vector is accessible here because we are using a classical simulator.

a qubit is not just an unknown ordinary bit. its amplitudes can interfere in later operations. that is the mechanism we need to understand next.

08 - gates & interference

a quantum gate changes amplitudes using a mathematical transformation. it preserves total probability. a circuit is a sequence of these transformations.

the hadamard gate is a useful example

```
new_a = (a+b)/sqrt(2)
new_b = (a-b)/sqrt(2)
```

start with a=1,b=0, meaning certain outcome 0. one hadamard produces a=b=1/sqrt(2), half probability each. applying it again gives a=1,b=0. the two amplitude contributions to outcome 1 cancel.

starting with a=1/sqrt(2),b=-1/sqrt(2), the same gate gives a=0,b=1. the sign changed the later outcome completely, even though both starting states had identical immediate 50/50 measurement probabilities. this is interference.

phase

phase is the direction of a complex amplitude. +1, -1 & i have the same squared magnitude but different directions. a relative phase is a difference between amplitudes. later gates can turn those differences into different probabilities.

what qaoa does with that

its cost layer changes phases according to candidate energy. that layer alone does not change the probabilities of measuring each basis string. the mixer then combines amplitudes, allowing those phase differences to affect probabilities. tuning their angles can favor some candidates, without guaranteeing the optimum.

a phase shared by every amplitude is a global phase & does not affect measurement probabilities. this is why we can keep a constant energy offset outside circuit evolution, while restoring it for reported energies.

09 - z, zz & the ising conversion

an operator is a mathematical transformation. for our diagonal energy operator, each basis string has a definite score. the name hamiltonian means energy operator here; it is not another optimization algorithm.

the z operator

bit value	z eigenvalue
0	+1
1	-1

an eigenvalue is the multiplier returned when the operator acts on one of its basis states. if z is the eigenvalue, the original bit equals (1-z)/2. try it - z=+1 gives bit 0; z=-1 gives bit 1.

```
bit = (1-Z)/2

bit_j*bit_k = (1-Z_j-Z_k+Z_j*Z_k)/4
```

a coefficient L multiplying one bit contributes +L/2 to the constant & -L/2 to its Z term. a pair coefficient Q contributes +Q/4 to the constant, -Q/4 to each Z term & +Q/4 to the ZZ term.

what zz means

multiply the two Z eigenvalues. equal bits give +1; different bits give -1. a weighted sum of Z & ZZ terms can therefore represent every single-bit & pair contribution in our polynomial.

binary_to_ising performs this substitution. cost_operator packages the results into SparsePauliOp. sparse means we list only needed operator terms instead of writing a full 512-by-512 matrix by hand. no new scoring rule is introduced.

10 - the qaoa circuit

qaoa means quantum approximate optimization algorithm. it is a method that combines a parameterized quantum circuit with an ordinary classical optimizer. approximate means the answer may be worse than the exact optimum.

the library template

```
QAOAAnsatz(cost_operator=op, reps=1, flatten=True)
```

an ansatz is a template with adjustable parameters. p, called reps in code, counts repeated cost/mixer layers. p=1 gives one of each & two angles, beta & gamma. this p is unrelated to the uppercase penalty P used in the energy formula.

the library starts with hadamard gates on the nine qubits, giving equal-magnitude amplitudes over all basis strings. it uses the default X mixer. we reuse those defaults rather than building custom gates.

what the angles do

gamma controls how strongly energy changes the relative phases. mathematically, a candidate with energy E gets a phase factor exp(-i*gamma*E). beta controls the mixer rotation. scipy tries different numerical values for these two angles.

the template is built once, then angles are bound repeatedly. flatten=True avoids nested circuit overhead during binding; it does not change the optimization problem.

why the library parameter order is saved

the code reads circuit.parameters & pairs that ordered collection with the numeric vector. it records parameter names in the report. guessing whether gamma or beta comes first could bind the right numbers to the wrong operations.

11 - optimizing is different from sampling

the optimizer needs one number to judge an angle choice. our objective is expected energy - the weighted average over all candidate energies, using their measurement probabilities.

```
expected_energy = sum(probability[s]*energy[s] for s in states)
```

a hypothetical distribution with 50% energy -2 & 50% energy 6 has average energy 2. its best candidate has energy -2. those are different measurements. qaoa optimizes the average, then we separately inspect sampled feasible candidates.

one objective call

```
bound = circuit.assign_parameters(dict(zip(params, angles)))
state = Statevector.from_instruction(bound)
energy = float(state.expectation_value(op).real + offset)
```

bind angles, simulate the circuit, calculate average operator energy & restore the constant. .real extracts the real component; this energy should be real. float makes it convenient to serialize.

bounded classical optimization

COBYLA changes the two angles without requiring derivatives. this run permits at most 30 objective calls & one seeded initial vector. it does not run 30 different student profiles. reaching the limit can return optimizer_success=false, even when the final circuit produces feasible samples.

final measurements

bind the returned angles, simulate again, seed the sampler & draw 1024 shots. the extra state construction is final sampling preparation, not another objective evaluation. keep the actual counts & filter invalid samples. a failed sample set stays an explicit failure, never the exact solver in disguise.

12 - reading your actual result

measurement	observed value
exact allocation	t1=0, t2=60, t3=30
sampled allocation	t1=0, t2=30, t3=0
exact utility	1.4166666666666665
sample utility	0.6666666666666666
absolute gap	0.7499999999999999
feasible measurements	86/1024 = 8.3984%
objective calls	30, stopped at cap
source commit	2a74c42

the feasible sampled plan uses one block for T2 & encodes two unused blocks in slack. leaving time unused is allowed. here it is suboptimal because valuable additional blocks existed, but they were not part of the best feasible sample observed.

the report records all samples, the optimization trace, parameters, package versions & durations. the quantum duration includes operator construction, optimization, sampling & decoding. simulation time is not hardware runtime.

why no victory claim

the exact algorithm is the correct baseline for this tiny problem. this particular qaoa attempt found a worse answer & required more local time. one simulated run cannot establish reliability, speedup, quantum advantage or improved student learning.

13 - workflow, replay & remaining gaps

a branch groups related changes. a commit records a source snapshot. a report generated from uncommitted code cannot be reproduced from its HEAD commit alone. the r4 run therefore happened on a clean source tree & records commit 2a74c42.

live run

```
python3 -m src.optimizer.quantum_demo --profile runtime/student_profile.json
```

reads a validated saved profile, runs exact & qaoa, writes new evidence. runtime is ignored by git because student data & generated files should not be staged accidentally.

saved replay

```
python3 -m evidence.risi.replay_comparison
```

audits & prints the saved fictional comparison. it says cached_replay & retains the original profile. it does not run qaoa or silently use newer app answers. the original report remains labelled live because it records a historical live simulation.

what is complete locally

the optimizer math, bounded simulation, profile comparison & report audit work. scoring & snapshot checks pass. the current branch has an empty practice-ranking module & empty app entry point. six retrieval checks are skipped & the smoke command exits incomplete at retrieval. that is not a complete integrated v2 product.

what you must personally do

read the guide, run the replay, explain the result in your own words & rehearse. independent review, event submission details & a real recording cannot be claimed just because an assistant generated files. the evidence records ai assistance explicitly.

14 - your presentation & self-check

a 60-second explanation

"examace uses a six-question diagnostic & a time budget to suggest study allocations. i implemented an exact planner over 27 possible allocations. we then expressed the same objective & constraints as a nine-bit energy model, with penalties for invalid decisions. i adapted a one-layer qaoa library circuit & simulated it locally. both solvers used the same fictional exported profile. the exact utility was 1.416667; the best feasible sampled utility was 0.666667. 86 of 1024 samples were feasible. the optimizer hit its 30-evaluation cap. this experiment demonstrates our mapping & feasibility checks; it does not show quantum advantage or learning gains."

answer these before presenting

why does the second block earn half utility? - it is the supplied diminishing-return heuristic.

why nine qubits? - six block decisions & three slack bits.

why square the budget error? - any nonzero error gets a positive penalty.

what is a coefficient? - a multiplier of one bit or a pair product.

why keep an offset? - preserve numerical energy equality, though it cannot change the optimum.

what does p=1 mean? - one cost/mixer layer, not one qubit or one shot.

what are 30 evaluations? - 30 checks of expected energy while adjusting angles.

why reject a plan that fits the minutes? - its second-block order or slack equality could still be wrong.

did qaoa win? - no, it returned worse utility on this run.

is the app finished? - not on this branch; ranking & ui integration remain blocked.

next reading

the appendices now show the actual source & checks. each important statement has its decision explained immediately before its numbered code line. obvious loop/branch scaffolding is included for completeness. do not memorize syntax; practice tracing one valid & one invalid candidate.

15 - sources & glossary

library ansatz & defaults

https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.QAOAAnsatz

statevector simulation & sampling

https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Statevector

adapted qaoa workflow

https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorithm

classical optimizer evaluation limit

https://docs.scipy.org/doc/scipy/reference/optimize.minimize-cobyla.html

the study-allocation formulas come from docs/01_requirements.md section 7. the worked arithmetic & explanations apply those formulas to the project code. the hardware tutorial is adapted to a local simulator, with one layer & the project cost operator. no hardware credentials are needed.

word	plain meaning
feasible	obeys every constraint
optimal	best feasible utility under the model
slack	unused whole blocks encoded in bits
coefficient	number multiplying a variable or product
offset	constant added to every energy
amplitude	complex number that determines probability
phase	direction/sign of a complex amplitude
interference	amplitudes combine & may cancel
ansatz	adjustable circuit template
expectation	probability-weighted average
shot	one sampled measurement

code appendix contracts.py

actual source - src/contracts.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

only the three planner records are included here. the later course-content records belong to the scoring pipeline. the repeated allocation validator is the existing warning recorded in release notes.

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
001  from pydantic import BaseModel, Field, field_validator
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
002  from typing import Annotated, Literal
```

decision - define a shared record whose fields are checked by pydantic.

```
004  class PlanRequest(BaseModel):
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
005      budget_minutes: int = Field(ge = 0, le = 180, strict = True)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
006      proficiency: dict[str, Annotated[float, Field(ge=0, le=1, allow_inf_nan=False,
strict=True)]]
007      requested_solver : Literal["exact", "qaoa"]
009      @field_validator("proficiency")
010      @classmethod
```

decision - group this operation behind a reusable function.

```
011      def check_topic_keys(cls, value):
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
012          expected = {"T1", "T2", "T3"}
013          if set(value) != expected:
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
014              raise ValueError("proficiency must contain exactly T1, T2, and T3")
```

decision - return the computed value or assembled result to the caller.

```
015          return value
```

decision - define a shared record whose fields are checked by pydantic.

```
018  class QuantumMetrics(BaseModel):
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
019      qubits: int = Field(gt=0, strict=True)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
020      p: int = Field(gt=0, strict=True)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
021      shots: int = Field(gt=0, strict=True)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
022      seed: int = Field(ge=0, strict=True)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
023      feasible_shot_rate: float = Field(ge=0, le=1, allow_inf_nan=False, strict=True)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
024      raw_best_utility: float | None = Field(default=None, ge=0, allow_inf_nan=False, strict=True)
```

decision - define a shared record whose fields are checked by pydantic.

```
026  class PlanResult(BaseModel):
027      allocations: dict[str, Literal[0, 30, 60]]
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
028      used_minutes: int = Field(ge=0, strict=True)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
029      unused_minutes: int = Field(ge=0, strict=True)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
030      utility: float = Field(ge=0, allow_inf_nan=False, strict=True)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
031      feasible: bool = Field(strict=True)
032      actual_solver: Literal["exact", "qaoa"]
```

decision - explicitly label this display as saved evidence, not a new circuit run.

```
033      execution_mode: Literal["live", "cached_replay"]
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
034      elapsed_ms: float = Field(ge=0, allow_inf_nan=False, strict=True)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
035      warnings: list[str] = Field(default_factory=list)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
036      quantum_metrics: QuantumMetrics | None = None
038      @field_validator("allocations")
039      @classmethod
```

decision - group this operation behind a reusable function.

```
040      def check_allocation_keys(cls, value):
041          if set(value) != {"T1", "T2", "T3"}:
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
042              raise ValueError("allocations must contain exactly T1, T2, and T3")
```

decision - return the computed value or assembled result to the caller.

```
043          return value
047      @field_validator("allocations")
048      @classmethod
```

decision - group this operation behind a reusable function.

```
049      def check_allocation_keys(cls, value):
```

```
050          if set(value) != {"T1", "T2", "T3"}:
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
051              raise ValueError("allocations must contain exactly T1, T2, and T3")
```

decision - return the computed value or assembled result to the caller.

```
052          return value
```

code appendix model.py

actual source - src/optimizer/model.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

```
001  """calculations used by study planner"""
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
002  import math
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
003  from numbers import Real
```

decision - fix topic order so tie decisions do not depend on dictionary order.

```
005  TOPICS = ("T1", "T2", "T3")
```

decision - reject invalid numeric inputs before they can corrupt an energy or comparison.

```
007  def validate(val, name, min, max):
008      """return a number within allowed range"""
009      if isinstance(val, bool) or not isinstance(val, Real):
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
010          raise ValueError(f"{name} must be a number")
```

decision - normalize accepted numbers to a floating-point value.

```
011      num = float(val)
012      if not math.isfinite(num):
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
013          raise ValueError(f"{name} must be finite")
014      if not min <= num <= max:
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
015          raise ValueError(
016              f"{name} must not exceed min and max"
017          )
```

decision - return the computed value or assembled result to the caller.

```
018      return num
```

decision - keep the exact & quantum sides on the same weight*(1-score) rule.

```
021  def topic_priority(weight, proficiency):
022      # dummy model
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
023      weight = validate(weight, "Topic weight", 0.5, 1.0)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
024      proficiency = validate(proficiency, "Proficiency", 0.0, 1.0)
```

decision - calculate remaining study priority using the shared heuristic.

```
025      return weight*(1-proficiency)
```

decision - add the utility of a proposed time allocation without selecting a plan.

```
027  def plan_utility(allocations, topic_weights, proficiency):
028      """calculates total utility(value) of a proposed study plan"""
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
029      total = 0.0
031      for topic_id, minutes in allocations.items():
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
032          priority = topic_priority(
033              topic_weights[topic_id],
034              proficiency[topic_id],
035          )
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
037          total += topic_utility(priority, minutes)
```

decision - return the computed value or assembled result to the caller.

```
039      return total
```

decision - encode the supplied diminishing returns for 0, 30 & 60 minutes.

```
041  def topic_utility(priority, minutes):
042      if minutes == 0:
```

decision - return the computed value or assembled result to the caller.

```
043          return 0.0
044      if minutes == 30:
```

decision - return the computed value or assembled result to the caller.

```
045          return priority
046      if minutes == 60:
047          # diminishing returns
```

decision - add the first block value & half that value for the second block.

```
048          return priority+priority*0.5
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
049      raise ValueError("minutes must be 0, 30, or 60")
```

decision - provide one reusable check for legal displayed time allocations.

```
052  def is_feasible(allocations, budget):
053      """feasibility change for extra protection/ plan obeys given constraints"""
054      if isinstance(budget, bool) or not isinstance(budget, int):
```

decision - return the computed value or assembled result to the caller.

```
055          return False
056      if not 0 <= budget <= 180:
```

decision - return the computed value or assembled result to the caller.

```
            return False
```

```
        if set(allocations) != set(TOPICS):
```

decision - return the computed value or assembled result to the caller.

```
            return False
```

```
        for mins in allocations.values():
```

```
            if isinstance(mins, bool) or not isinstance(mins, int):
```

decision - return the computed value or assembled result to the caller.

```
                    return False
```

```
            if mins not in (0, 30, 60):
```

decision - return the computed value or assembled result to the caller.

```
                return False
```

decision - return the computed value or assembled result to the caller.

```
        return sum(allocations.values()) <= budget
```

```
    # bit order fixed, slack bits start with lowest value
```

decision - freeze the meaning of every bit position for files, operators & decoders.

```
    BIT_ORDER = ("x11", "x12", "x21", "x22", "x31", "x32", "y0", "y1", "y2")
```

decision - reject malformed nine-bit candidates before indexing them.

```
    def _check_bits(bits):
```

```
        if len(bits) != 9 or any(type(bit) is not int or bit not in (0, 1) for bit in bits):
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
            raise ValueError("expected 9 integer bits in bit_order")
```

decision - check topic keys & derive priorities from validated scores.

```
    def _profile_priorities(weights, request):
```

```
        if set(weights) != set(TOPICS):
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
            raise ValueError("weights must contain exactly T1, T2, T3")
```

decision - return the computed value or assembled result to the caller.

```
        return [topic_priority(weights[t], request.proficiency[t]) for t in TOPICS]
```

decision - keep a direct reference expression against which conversion code can be checked.

```
    def binary_energy(bits, weights, request):
```

```
        """calc energy directly from the given formula"""
```

```
        _check_bits(bits)
```

decision - derive the same three priorities from this profile.

```
        vals = _profile_priorities(weights, request)
```

decision - make any integer constraint violation more expensive than total possible utility.

```
        penalty = 1 + 1.5 * sum(vals)
```

decision - discard the partial-block remainder; only whole 30-minute blocks can be selected.

```
089      blocks = request.budget_minutes // 30
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
090      util = 0.0
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
091      bad_order = 0
092      for i, val in enumerate(vals):
```

decision - read the current topic's adjacent pair of decision bits.

```
093          first, second = bits[2 * i:2 * i + 2]
```

decision - add the value selected by these first/second decision bits.

```
094          util += val * first + 0.5 * val * second
```

decision - count a violation only when second is 1 & first is 0.

```
095          bad_order += second * (1 - first)
```

decision - decode unused blocks with binary place values 1, 2 & 4.

```
096      slack = bits[6] + 2 * bits[7] + 4 * bits[8]
```

decision - calculate the equality error that must be zero for a valid candidate.

```
097      budget_diff = sum(bits[:6]) + slack - blocks
```

decision - minimize negative utility plus positive constraint penalties.

```
098      return -util + penalty * (budget_diff ** 2 + bad_order)
```

decision - expand the energy into a constant, single-bit coefficients & pair coefficients.

```
101  def build_binary_model(weights, request):
102      """build constant, linear and pair coeffs"""
```

decision - derive the same three priorities from this profile.

```
103      vals = _profile_priorities(weights, request)
```

decision - make any integer constraint violation more expensive than total possible utility.

```
104      penalty = 1 + 1.5 * sum(vals)
```

decision - discard the partial-block remainder; only whole 30-minute blocks can be selected.

```
105      blocks = request.budget_minutes // 30
```

decision - assign block-budget coefficients to study bits & weighted slack bits.

```
106      counts = (1, 1, 1, 1, 1, 1, 1, 2, 4)
```

decision - associate utility with study bits & zero utility with slack bits.

```
107      values = [value for a in vals for value in (a, 0.5 * a)] + [0.0] * 3
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
108      linear = []
109      for j in range(9):
```

decision - collect the squared-budget linear term, budget cross term & negative utility.

```
110          coeff = penalty * counts[j] ** 2 - 2 * penalty * blocks * counts[j] - values[j]
111          if j in (1, 3, 5):
```

decision - add the positive second-block ordering contribution.

```
            coeff += penalty
```

decision - store one single-bit coefficient at its bit index.

```
        linear.append(coeff)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    quadratic = {}
    for j in range(9):
        for k in range(j + 1, 9):
```

decision - collect this pair's cross term from the expanded budget square.

```
            coeff = 2 * penalty * counts[j] * counts[k]
            if (j, k) in ((0, 1), (2, 3), (4, 5)):
```

decision - subtract the same-topic ordering pair contribution.

```
                coeff -= penalty
```

decision - store this pair once under its ordered index tuple.

```
            quadratic[j, k] = coeff
```

decision - return the computed value or assembled result to the caller.

```
    return {
```

decision - retain the constant term for numerical energy equality.

```
        "constant": penalty * blocks ** 2,
```

decision - save this named field in the result so later code can inspect or audit it.

```
        "linear": linear,
```

decision - save this named field in the result so later code can inspect or audit it.

```
        "quadratic": quadratic,
    }
```

decision - evaluate the expanded form independently of the direct formula.

```
def polynomial_energy(bits, model):
    """calc binary energy, constant included"""
    _check_bits(bits)
```

decision - return the computed value or assembled result to the caller.

```
    return (
        model["constant"]
        + sum(coeff * bits[j] for j, coeff in enumerate(model["linear"]))
        + sum(coeff * bits[j] * bits[k] for (j, k), coeff in model["quadratic"].items())
    )
```

decision - replace binary variables with z eigenvalues while preserving full energy.

```
def binary_to_ising(model):
    """convert binary to ising, keep offset"""
```

decision - each linear binary term contributes negative half its coefficient to z.

```
    linear = [-coeff / 2 for coeff in model["linear"]]
```

decision - start the ising constant with the binary constant & half the linear coefficients.

```
    offset = model["constant"] + sum(model["linear"]) / 2
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    quadratic = {}
    for (j, k), coeff in model["quadratic"].items():
```

decision - a binary pair expands into four terms, each scaled by one quarter.

```
        coupling = coeff / 4
```

decision - store this pair once under its ordered index tuple.

```
        quadratic[j, k] = coupling
```

decision - subtract the pair contribution from its first z coefficient.

```
        linear[j] -= coupling
```

decision - subtract the pair contribution from its second z coefficient.

```
        linear[k] -= coupling
```

decision - restore the constant contribution created by this pair substitution.

```
        offset += coupling
```

decision - return the computed value or assembled result to the caller.

```
    return {"offset": offset, "linear": linear, "quadratic": quadratic}
```

decision - check the converted coefficients on ordinary bitstrings.

```
def ising_energy(bits, model):
    """calc ising energy, 0 maps to +1 and 1 maps to -1"""
    _check_bits(bits)
```

decision - map measured bit 0 to +1 & measured bit 1 to -1.

```
    spins = [1 - 2 * bit for bit in bits]
```

decision - return the computed value or assembled result to the caller.

```
    return (
        model["offset"]
        + sum(coeff * spins[j] for j, coeff in enumerate(model["linear"]))
        + sum(coeff * spins[j] * spins[k] for (j, k), coeff in model["quadratic"].items())
    )
```

decision - translate a sampled candidate & flag violations without repairing it.

```
def decode_bits(bits, weights, request):
    """decode bits and check constraints"""
    _check_bits(bits)
```

decision - derive the same three priorities from this profile.

```
    vals = _profile_priorities(weights, request)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    allocs = {}
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    util = 0.0
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    warnings = []
    for i, topic in enumerate(TOPICS):
```

decision - read the current topic's adjacent pair of decision bits.

```
        first, second = bits[2 * i:2 * i + 2]
```

decision - convert selected block bits into topic minutes.

```
        allocs[topic] = 30 * (first + second)
```

decision - add the value selected by these first/second decision bits.

```
        util += vals[i] * (first + 0.5 * second)
        if second > first:
```

decision - keep a concrete reason why this candidate is invalid.

```
            warnings.append(f"{topic}: second block needs first block")
```

decision - decode unused blocks with binary place values 1, 2 & 4.

```
    slack = bits[6] + 2 * bits[7] + 4 * bits[8]
    if sum(bits[:6]) + slack != request.budget_minutes // 30:
```

decision - keep a concrete reason why this candidate is invalid.

```
        warnings.append("selected blocks + slack must equal budget blocks")
    if not is_feasible(allocs, request.budget_minutes):
```

decision - keep a concrete reason why this candidate is invalid.

```
        warnings.append("allocs exceed time constraints")
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    used = sum(allocs.values())
```

decision - return the computed value or assembled result to the caller.

```
    return {
```

decision - save this named field in the result so later code can inspect or audit it.

```
        "allocations": allocs,
```

decision - save this named field in the result so later code can inspect or audit it.

```
        "used_minutes": used,
```

decision - save this named field in the result so later code can inspect or audit it.

```
        "unused_minutes": request.budget_minutes - used,
```

decision - save this named field in the result so later code can inspect or audit it.

```
        "utility": util,
```

decision - declare feasibility only if every decoder constraint passed.

```
        "feasible": not warnings,
```

decision - save this named field in the result so later code can inspect or audit it.

```
        "slack_blocks": slack,
```

decision - save this named field in the result so later code can inspect or audit it.

```
        "warnings": warnings,
    }
```

decision - restore qubit-index order from qiskit displayed strings.

```
def qiskit_bitstring_to_bits(bitstr):
    """reverse displayed bits into qubit order"""
    if not isinstance(bitstr, str) or len(bitstr) != 9 or set(bitstr) - {"0", "1"}:
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
        raise ValueError("expected 9 binary characters")
```

decision - reverse the displayed string & convert characters to integer bits.

```
    return [int(bit) for bit in reversed(bitstr)]
```

code appendix classical.py

actual source - src/optimizer/classical.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

```
001  """generates every possible plan"""
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
002  from itertools import product
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
003  from time import perf_counter
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
005  from src.optimizer.model import plan_utility, is_feasible, TOPICS
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
006  from src.contracts import PlanRequest, PlanResult
```

decision - try every legal topic-time combination & retain the best feasible plan.

```
008  def solve_exact(weights, proficiency, budget):
009      """find, by brute force, the highest utility allocation within budget given"""
```

decision - start elapsed timing before operator/model construction.

```
010      start = perf_counter()
```

decision - reuse the fixed order for allocations & tie comparisons.

```
011      topic_ids = TOPICS
013      # validation
014      if isinstance(budget, bool) or not isinstance(budget, int):
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
015          raise ValueError("budget must be an integer number in minutes")
016      if not 0 <= budget <= 180:
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
017          raise ValueError("budget must be between 0 and 180")
018      if set(weights) != set(topic_ids):
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
019          raise ValueError("Weights must contain exactly T1, T2, T3")
020      if set(proficiency) != set(topic_ids):
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
021              raise ValueError("Proficiency must contain exactly T1, T2, T3")
024      # set up
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
025      best_allocs = None
```

decision - start below any real candidate utility, so the first feasible plan can win.

```
026      best_util = float("-inf")
```

decision - prefer less used time, then larger earlier-topic allocations.

```
027      best_tie = None

029      for mins in product((0, 30, 60), repeat=3):
```

decision - pair topic names with the candidate minute tuple.

```
030          allocs = dict(zip(topic_ids, mins))
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
031          used_mins = sum(mins)

032          # skip plan if not feasible

033          if not is_feasible(allocs, budget):

034               continue
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
036          util = plan_utility(allocs, weights, proficiency)
```

decision - prefer less used time, then larger earlier-topic allocations.

```
037          tie = (-used_mins, *mins)

038          # accounting for floating point issues, values are subject to change
```

decision - require a utility improvement larger than rounding tolerance.

```
039          better_util = util > best_util + 1e-12
```

decision - treat negligible floating-point differences as a tie.

```
040          equal_util = abs(util-best_util) <= 1e-12

042          if best_allocs is None or better_util or (equal_util and tie > best_tie):
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
043              best_allocs = allocs
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
044              best_util = util
```

decision - prefer less used time, then larger earlier-topic allocations.

```
045              best_tie = tie
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
047      used_mins = sum(best_allocs.values())
```

decision - return the computed value or assembled result to the caller.

```
049      return {
```

decision - save this named field in the result so later code can inspect or audit it.

```
050          "allocations": best_allocs,
```

decision - save this named field in the result so later code can inspect or audit it.

```
051          "used_minutes": used_mins,
```

decision - save this named field in the result so later code can inspect or audit it.

```
052          "unused_minutes": budget - used_mins,
```

decision - save this named field in the result so later code can inspect or audit it.

```
053          "utility": best_util,
```

decision - save this named field in the result so later code can inspect or audit it.

```
054          "feasible": is_feasible(best_allocs, budget),
```

decision - save this named field in the result so later code can inspect or audit it.

```
055          "actual_solver": "exact",
```

decision - save this named field in the result so later code can inspect or audit it.

```
056          "execution_mode": "live",
```

decision - save this named field in the result so later code can inspect or audit it.

```
057          "elapsed_ms": (perf_counter() - start) * 1000,
```

decision - save this named field in the result so later code can inspect or audit it.

```
058          "warnings": [],
```

decision - save this named field in the result so later code can inspect or audit it.

```
059          "quantum_metrics": None,
060      }
```

decision - adapt the shared request object to the exact calculation & shared result object.

```
062  def solve_plan(weights: dict[str, float], request: PlanRequest) -> PlanResult:
063      """completes an exact-planning rq"""
064      if request.requested_solver != "exact":
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
065            raise ValueError("planner supports only the exact solver")
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
066      result = solve_exact(weights, request.proficiency, request.budget_minutes)
```

decision - expand dictionary fields into the shared validated output model.

```
067      return PlanResult(**result)
```

code appendix quantum_demo.py

actual source - src/optimizer/quantum_demo.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

```
001  """bounded qaoa simulation, separate from the app"""
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
003  import argparse
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
004  import json
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
005  import subprocess
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
006  from datetime import datetime, timezone
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
007  from importlib.metadata import version
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
008  from pathlib import Path
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
009  from time import perf_counter
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
011  import numpy as np
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
012  from qiskit.circuit.library import QAOAAnsatz
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
013  from qiskit.quantum_info import SparsePauliOp, Statevector
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
014  from scipy.optimize import minimize
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
016  from src.intelligence.validate import validate_snapshot
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
017  from src.optimizer.classical import solve_plan
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
018  from src.optimizer.model import (
019      BIT_ORDER, TOPICS, binary_to_ising, build_binary_model,
020      decode_bits, qiskit_bitstring_to_bits,
021  )
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
023  SEED = 7
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
024  SHOTS = 1024
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
025  MAX_EVALS = 30
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
026  REF = "https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorithm"
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
027  ROOT = Path(__file__).resolve().parents[2]
```

decision - reuse alex's snapshot validation at the file boundary.

```
030  def load_profile(path):
031      """load & validate the exported snapshot"""
```

decision - turn a file or dictionary into the shared checked snapshot.

```
032      return validate_snapshot(json.loads(Path(path).read_text()))
```

decision - build the qiskit energy operator from r2 coefficients.

```
035  def cost_operator(model):
036      """make z & zz terms, return offset separately"""
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
037      terms = []
038      for j, coeff in enumerate(model["linear"]):
```

decision - add a single-qubit energy contribution at its explicit qubit index.

```
039          terms.append(("Z", [j], coeff))
040      for (j, k), coeff in model["quadratic"].items():
```

decision - add the two-qubit interaction contribution at its two indices.

```
041          terms.append(("ZZ", [j, k], coeff))
```

decision - construct a nine-qubit operator without manual string-endianness mistakes.

```
042      op = SparsePauliOp.from_sparse_list(terms, num_qubits=9).simplify()
```

decision - preserve the constant separately; it does not need circuit evolution.

```
043      return op, model["offset"]
```

decision - rank only feasible samples that actually occurred.

```
046  def select_sample(counts, weights, request):
047      """pick best feasible sample, same tie rule as exact"""
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
048      best = None
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
049      best_bits = None
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    feasible_shots = 0
    for bitstr, count in counts.items():
```

decision - convert displayed sample order before decoding.

```
        bits = qiskit_bitstring_to_bits(bitstr)
```

decision - check the sampled bits against ordering, slack & budget constraints.

```
        plan = decode_bits(bits, weights, request)
        if not plan["feasible"]:
            continue
```

decision - count measurements, not just distinct feasible strings.

```
        feasible_shots += count
```

decision - force the topic order used by the tie rule.

```
        mins = tuple(plan["allocations"][t] for t in TOPICS)
```

decision - prefer less used time, then larger earlier-topic allocations.

```
        tie = (-plan["used_minutes"], *mins)
        if best is not None:
```

decision - compare sampled utility with the current best utility.

```
            diff = plan["utility"] - best["utility"]
```

decision - force the topic order used by the tie rule.

```
            best_mins = tuple(best["allocations"][t] for t in TOPICS)
```

decision - prefer less used time, then larger earlier-topic allocations.

```
            best_tie = (-best["used_minutes"], *best_mins)
            if diff < -1e-12 or (abs(diff) <= 1e-12 and tie <= best_tie):
                continue
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
        best = plan
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
        best_bits = bits
```

decision - return only an observed feasible answer & its share of all measurements.

```
    return best, best_bits, feasible_shots / sum(counts.values())
```

decision - perform one bounded simulator attempt & report actual results.

```
def run_qaoa(snapshot):
    """one start, 30 evals max, 1024 final samples"""
```

decision - revalidate saved inputs before starting calculations.

```
    snapshot = validate_snapshot(snapshot)
```

decision - start elapsed timing before operator/model construction.

```
    start = perf_counter()
```

decision - change the solver label on a copy, preserving original inputs.

```
    request = snapshot.request.model_copy(update={"requested_solver": "qaoa"})
```

decision - expand the profile-specific cost, never a hard-coded student cost.

```
075     binary = build_binary_model(snapshot.topic_weights, request)
```

decision - convert those coefficients into z & zz form.

```
076     ising = binary_to_ising(binary)
```

decision - package the converted cost for qiskit while retaining the constant.

```
077     op, offset = cost_operator(ising)
```

decision - reuse the library circuit with one layer, default initial state & mixer.

```
078     circuit = QAOAAnsatz(cost_operator=op, reps=1, flatten=True)
```

decision - record actual parameter order so angles bind correctly.

```
079     params = tuple(circuit.parameters)
```

decision - create a local seeded random generator for repeatable initial angles.

```
080     rng = np.random.default_rng(SEED)
```

decision - choose one starting vector; no multiple-start tuning is performed.

```
081     initial = rng.uniform(0, np.pi, len(params))
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
082     trace = []
```

decision - give scipy a scalar expected energy for its current angle choice.

```
084     def objective(angles):
```

decision - bind the numeric angles to the circuit parameters.

```
085         bound = circuit.assign_parameters(dict(zip(params, angles)))
```

decision - simulate amplitudes of the bound circuit on a regular computer.

```
086         state = Statevector.from_instruction(bound)
```

decision - calculate real expected energy & add the saved constant offset.

```
087         energy = float(state.expectation_value(op).real + offset)
```

decision - record each objective call for the evaluation-budget audit.

```
088         trace.append({"params": [float(a) for a in angles], "energy": energy})
```

decision - return the computed value or assembled result to the caller.

```
089         return energy
```

decision - ask scipy to change angles to lower expected energy.

```
091     result = minimize(
```

decision - use the objective callback, single starting vector & derivative-free method.

```
092         objective, initial, method="COBYLA",
```

decision - cap this optimizer at 30 objective calls & suppress routine output.

```
093         options={"maxiter": MAX_EVALS, "disp": False},
094     )
```

decision - bind the numeric angles to the circuit parameters.

```
095     bound = circuit.assign_parameters(dict(zip(params, result.x)))
```

decision - simulate amplitudes of the bound circuit on a regular computer.

```
096     state = Statevector.from_instruction(bound)
```

decision - seed the measurement sampler for this final distribution.

```
097     state.seed(SEED)
```

decision - draw 1024 measurements & normalize count values for json.

```
098     counts = {key: int(val) for key, val in state.sample_counts(SHOTS).items()}
```

decision - filter final samples & choose the best feasible one actually measured.

```
099     best, bits, rate = select_sample(counts, snapshot.topic_weights, request)
```

decision - include construction, optimization, sampling & decoding in attempt duration.

```
100     elapsed = (perf_counter() - start) * 1000
```

decision - return the computed value or assembled result to the caller.

```
101     return {
```

decision - save this named field in the result so later code can inspect or audit it.

```
102         "actual_solver": "qaoa",
```

decision - save this named field in the result so later code can inspect or audit it.

```
103         "execution_mode": "live",
```

decision - save this named field in the result so later code can inspect or audit it.

```
104         "backend": "local statevector simulation",
```

decision - save this named field in the result so later code can inspect or audit it.

```
105         "qubits": circuit.num_qubits,
```

decision - save this named field in the result so later code can inspect or audit it.

```
106         "p": 1,
```

decision - save this named field in the result so later code can inspect or audit it.

```
107         "shots": SHOTS,
```

decision - save this named field in the result so later code can inspect or audit it.

```
108         "seed": SEED,
```

decision - save this named field in the result so later code can inspect or audit it.

```
109         "objective_evaluations": len(trace),
```

decision - save this named field in the result so later code can inspect or audit it.

```
110         "initial_params": initial.tolist(),
```

decision - save this named field in the result so later code can inspect or audit it.

```
111         "parameter_order": [str(param) for param in params],
```

decision - save this named field in the result so later code can inspect or audit it.

```
112         "params": result.x.tolist(),
```

decision - save this named field in the result so later code can inspect or audit it.

```
113         "expected_energy": float(result.fun),
```

decision - save this named field in the result so later code can inspect or audit it.

```
114         "offset": offset,
```

decision - save this named field in the result so later code can inspect or audit it.

```
115         "optimizer_success": bool(result.success),
```

decision - save this named field in the result so later code can inspect or audit it.

```
116          "optimizer_message": str(result.message),
```

decision - save this named field in the result so later code can inspect or audit it.

```
117          "best_sample": best,
```

decision - save this named field in the result so later code can inspect or audit it.

```
118          "best_bits": bits,
```

decision - keep utility null when no feasible sample exists.

```
119          "raw_best_utility": None if best is None else best["utility"],
```

decision - save this named field in the result so later code can inspect or audit it.

```
120          "feasible_shot_rate": rate,
```

decision - save this named field in the result so later code can inspect or audit it.

```
121          "elapsed_ms": elapsed,
```

decision - save this named field in the result so later code can inspect or audit it.

```
122          "status": "failed" if best is None else "feasible sample found",
```

decision - save this named field in the result so later code can inspect or audit it.

```
123          "warnings": ["no feasible sample"] if best is None else [],
```

decision - save this named field in the result so later code can inspect or audit it.

```
124          "trace": trace,
```

decision - save this named field in the result so later code can inspect or audit it.

```
125          "counts": counts,
126      }
```

decision - preserve common inputs, exact reference & observed quantum evidence.

```
129  def compare(snapshot):
130      """same profile for exact & qaoa, save original inputs"""
```

decision - revalidate saved inputs before starting calculations.

```
131      snapshot = validate_snapshot(snapshot)
```

decision - change the solver label on a copy, preserving original inputs.

```
132      exact_request = snapshot.request.model_copy(update={"requested_solver": "exact"})
```

decision - compute the exact baseline for the identical profile.

```
133      exact = solve_plan(snapshot.topic_weights, exact_request)
```

decision - run one bounded quantum simulation on that saved profile.

```
134      quantum = run_qaoa(snapshot)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
135      best_util = quantum["raw_best_utility"]
```

decision - read the actual source commit rather than accepting a typed label.

```
136      commit = subprocess.run(
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
137          ["git", "rev-parse", "HEAD"], cwd=ROOT,
```

decision - raise on git failure & read its output without noisy terminal messages.

```
138          check=True, capture_output=True, text=True,
```

```
139      ).stdout.strip()
```

decision - detect uncommitted edits so evidence does not imply a clean source snapshot.

```
140      dirty = bool(subprocess.run(
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
141          ["git", "status", "--porcelain"], cwd=ROOT,
```

decision - raise on git failure & read its output without noisy terminal messages.

```
142          check=True, capture_output=True, text=True,
```

```
143      ).stdout.strip())
```

decision - return the computed value or assembled result to the caller.

```
144      return {
```

decision - save this named field in the result so later code can inspect or audit it.

```
145          "schema_version": 1,
```

decision - save this named field in the result so later code can inspect or audit it.

```
146          "created_utc": datetime.now(timezone.utc).isoformat(),
```

decision - save this named field in the result so later code can inspect or audit it.

```
147          "execution_mode": "live",
```

decision - retain the entire original validated problem in the report.

```
148          "profile": snapshot.model_dump(mode="json"),
```

decision - save this named field in the result so later code can inspect or audit it.

```
149          "code_commit": commit,
```

decision - save this named field in the result so later code can inspect or audit it.

```
150          "working_tree_dirty": dirty,
```

decision - save this named field in the result so later code can inspect or audit it.

```
151          "reference": REF,
```

decision - save this named field in the result so later code can inspect or audit it.

```
152          "adaptation": "study allocation cost, p=1, local statevector, bounded cobyla",
```

decision - save this named field in the result so later code can inspect or audit it.

```
153          "bit_order": list(BIT_ORDER),
```

decision - record actual installed library versions for reproduction.

```
154          "versions": {name: version(name) for name in ("qiskit", "numpy", "scipy", "pydantic")},
```

decision - save this named field in the result so later code can inspect or audit it.

```
155          "exact": exact.model_dump(mode="json"),
```

decision - save this named field in the result so later code can inspect or audit it.

```
156          "quantum": quantum,
```

decision - record absolute difference from exact; null remains failure, not a substituted answer.

```
157          "absolute_utility_gap": None if best_util is None else abs(exact.utility - best_util),
```

decision - save this named field in the result so later code can inspect or audit it.

```
158          "limitations": [
159              "one fictional profile, local simulation",
160              "no hardware timing, quantum advantage or learning gain claim",
161              "uncommitted code recorded separately from code_commit" if dirty else "committed
code",
162          ],
163      }
```

decision - connect the command line to the implementation without importing the ui.

```
166  def main():
```

decision - define the supported terminal command & its help text.

```
167      parser = argparse.ArgumentParser(description="compare exact & qaoa on an exported profile")
```

decision - declare an input/output path option rather than hard-coding a student file.

```
168      parser.add_argument("--profile", type=Path, required=True)
```

decision - declare an input/output path option rather than hard-coding a student file.

```
169      parser.add_argument("--output", type=Path, default=ROOT / "runtime/quantum_comparison.json")
```

decision - read the user's supplied command arguments.

```
170      args = parser.parse_args()
171      try:
```

decision - validate the file, then compute both solver results.

```
172          report = compare(load_profile(args.profile))
173      except (ValueError, OSError) as error:
```

decision - return a readable failing terminal status for bad input.

```
174          parser.exit(1, f"quantum run failed - {error}\n")
```

decision - create missing report folders without deleting anything.

```
175      args.output.parent.mkdir(parents=True, exist_ok=True)
```

decision - save strict json; non-finite numbers are rejected.

```
176      args.output.write_text(json.dumps(report, indent=2, allow_nan=False) + "\n")
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
177      quantum = report["quantum"]
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
178      print(f"exact util - {report['exact']['utility']:.6f}")
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
179      print(f"sample util - {quantum['raw_best_utility']}")
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
180      print(f"feasible shots - {quantum['feasible_shot_rate']:.3f}")
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
181      print(f"evals - {quantum['objective_evaluations']}")
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
182      print(f"report - {args.output}")
```

decision - signal a failed run to scripts when no feasible sample was found.

```
183      return 1 if quantum["best_sample"] is None else 0

186  if __name__ == "__main__":
```

decision - return the program result to the terminal.

```
187      raise SystemExit(main())
```

code appendix
export_fictional_profile.py

actual source - evidence/risi/export_fictional_profile.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

```
001  """export a fictional diagnostic through the production pipeline"""
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
002  import json
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
003  from pathlib import Path
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
005  from src.contracts import PlanRequest
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
006  from src.intelligence.assess import score_diagnostic
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
007  from src.intelligence.pipeline import load_course, make_snapshot
```

decision - connect the command line to the implementation without importing the ui.

```
010  def main():
```

decision - locate the repository from this script rather than assuming the caller's directory.

```
011      root = Path(__file__).resolve().parents[2]
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
012      data = root / "data/demo"
```

decision - read a reviewed fictional answer set instead of hard-coded scores.

```
013      case = json.loads((data / "student_cases.json").read_text())["cases"][0]
```

decision - validate the reviewed course before scoring.

```
014      course = load_course(data / "course.json")
```

decision - derive topic scores from actual submitted choice ids.

```
015      diagnostic = score_diagnostic(course, case["submitted_choices"])
```

decision - build the shared validated planning input.

```
016      request = PlanRequest(
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
017          budget_minutes=case["budget_minutes"],
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
018          proficiency=diagnostic.proficiency,
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
019          requested_solver="exact",
```

```
020      )
```

decision - derive actual topic weights & preserve the submitted scores/budget.

```
021      snapshot = make_snapshot(course, request)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
022      output = root / "runtime/student_profile.json"
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
023      output.parent.mkdir(parents=True, exist_ok=True)
```

decision - export a reusable validated profile file.

```
024      output.write_text(snapshot.model_dump_json(indent=2) + "\n")
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
025      print(f"fictional profile - {output}")
028  if __name__ == "__main__":
029      main()
```

code appendix replay_comparison.py

actual source - evidence/risi/replay_comparison.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

```
001  """audit & show saved evidence, no new quantum run"""
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
002  import argparse
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
003  import json
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
004  from math import isclose, isfinite
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
005  from pathlib import Path
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
007  from src.intelligence.validate import validate_snapshot
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
008  from src.optimizer.classical import solve_plan
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
009  from src.optimizer.quantum_demo import select_sample
```

decision - check saved evidence before replaying it.

```
012  def audit_report(report):
013      """reject changed inputs, mismatched results or incomplete evidence"""
```

decision - revalidate saved inputs before starting calculations.

```
014      snapshot = validate_snapshot(report["profile"])
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
015      quantum = report["quantum"]
```

decision - compute the exact baseline for the identical profile.

```
016      exact = solve_plan(snapshot.topic_weights, snapshot.request.model_copy(update={"requested_so
lver": "exact"}))
```

decision - fail an audit clearly when one relationship is inconsistent.

```
017      def check(condition, message):
018          if not condition:
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
019              raise ValueError(message)
```

decision - verify this saved field agrees with independently recomputed evidence.

```
020      check(report["execution_mode"] == quantum["execution_mode"] == "live", "original evidence
must be a live run")
```

decision - verify this saved field agrees with independently recomputed evidence.

```
021      check(report["working_tree_dirty"] is False, "evidence source was uncommitted")
```

decision - verify this saved field agrees with independently recomputed evidence.

```
022      check(len(report["code_commit"]) == 40, "missing source commit")
```

decision - verify this saved field agrees with independently recomputed evidence.

```
023      check(quantum["qubits"] == 9 and quantum["p"] == 1, "wrong circuit size")
```

decision - verify this saved field agrees with independently recomputed evidence.

```
024      check(quantum["seed"] == 7 and quantum["shots"] == 1024, "wrong sampling settings")
```

decision - verify this saved field agrees with independently recomputed evidence.

```
025      check(0 < quantum["objective_evaluations"] == len(quantum["trace"]) <= 30, "wrong eval
budget")
```

decision - verify this saved field agrees with independently recomputed evidence.

```
026      check(sum(quantum["counts"].values()) == 1024, "wrong shot total")
```

decision - verify this saved field agrees with independently recomputed evidence.

```
027      check(all(type(n) is int and n > 0 for n in quantum["counts"].values()), "invalid counts")
```

decision - verify this saved field agrees with independently recomputed evidence.

```
028      check(exact.allocations == report["exact"]["allocations"], "exact allocation changed")
```

decision - verify this saved field agrees with independently recomputed evidence.

```
029      check(isclose(exact.utility, report["exact"]["utility"], abs_tol=1e-8), "exact utility
changed")
```

decision - filter final samples & choose the best feasible one actually measured.

```
030      best, bits, rate = select_sample(quantum["counts"], snapshot.topic_weights,
snapshot.request)
```

decision - verify this saved field agrees with independently recomputed evidence.

```
031      check(best == quantum["best_sample"] and bits == quantum["best_bits"], "saved sample
changed")
```

decision - verify this saved field agrees with independently recomputed evidence.

```
032      check(isclose(rate, quantum["feasible_shot_rate"], abs_tol=1e-12), "feasible rate changed")
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
033      util = None if best is None else best["utility"]
```

decision - verify this saved field agrees with independently recomputed evidence.

```
034      check(util == quantum["raw_best_utility"], "sample utility changed")
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
035      gap = None if util is None else abs(exact.utility-util)
```

decision - verify this saved field agrees with independently recomputed evidence.

```
036      check(gap == report["absolute_utility_gap"], "utility gap changed")
037      for result in (quantum, report["exact"]):
```

decision - verify this saved field agrees with independently recomputed evidence.

```
038          check(isfinite(result["elapsed_ms"]) and result["elapsed_ms"] >= 0, "invalid timing")
```

decision - return the computed value or assembled result to the caller.

```
039      return snapshot
```

decision - connect the command line to the implementation without importing the ui.

```
042  def main():
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
043      parser=argparse.ArgumentParser(description="replay saved fictional-profile comparison")
```

decision - declare an input/output path option rather than hard-coding a student file.

```
044      parser.add_argument("--report", type=Path, default=Path(__file__).with_name("r4_comparison.j
son"))
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
045      args=parser.parse_args()
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
046      report=json.loads(args.report.read_text())
```

decision - check saved evidence before displaying it as a replay.

```
047      snapshot=audit_report(report)
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
048      print("cached_replay - saved local simulation, no new run")
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
049      print(f"original run - {report['created_utc']}")
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
050      print(f"profile - {snapshot.request.model_dump()}")
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
051      print(f"exact - {report['exact']['allocations']}, util {report['exact']['utility']:.6f}")
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
052      print(f"sample - {report['quantum']['best_sample']}")
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
053      print(f"gap - {report['absolute_utility_gap']:.6f}")
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
054      print(f"feasible rate - {report['quantum']['feasible_shot_rate']:.6f}")

057  if __name__ == "__main__":

058      main()
```

code appendix test_intelligence.py

actual source - tests/test_intelligence.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

```
001  """risi checks for alex's scoring, validation & snapshot export"""
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
002  import copy
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
003  import json
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
004  from pathlib import Path
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
006  import pytest
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
008  from src.contracts import PlanRequest
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
009  from src.intelligence.assess import score_diagnostic
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
010  from src.intelligence.pipeline import load_course, make_snapshot
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
011  from src.intelligence.validate import validate_course, validate_snapshot
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
012  from src.intelligence import rank
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
014  DATA=Path(__file__).resolve().parents[1]/"data/demo"

017  @pytest.fixture
```

decision - group this operation behind a reusable function.

```
018  def course():
```

decision - return the computed value or assembled result to the caller.

```
019      return load_course(DATA/"course.json")
```

decision - run this behavior against several specified inputs rather than a single happy path.

```
022  @pytest.mark.parametrize("correct,expected", [(True,1.0),(False,0.0)])
```

decision - group this operation behind a reusable function.

```
023  def test_all_correct_or_wrong(course,correct,expected):
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
024      answers={}

025      for q in course.questions:
```

```
026          if q.purpose == "diagnostic":
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
027              answers[q.question_id]=q.correct_choice if correct else next(c for c in q.choices
if c != q.correct_choice)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
028      result=score_diagnostic(course,answers)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
029      assert result.proficiency == dict.fromkeys(("T1","T2","T3"),expected)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
030      assert result.submitted_choices == answers
```

decision - run this behavior against several specified inputs rather than a single happy path.

```
033  @pytest.mark.parametrize("case_index", [0,1])
```

decision - group this operation behind a reusable function.

```
034  def test_reviewed_fictional_scores(course,case_index):
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
035      case=json.loads((DATA/"student_cases.json").read_text())["cases"][case_index]
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
036      result=score_diagnostic(course,case["submitted_choices"])
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
037      assert result.proficiency == case["expected_with_test_weights"]["proficiency"]
```

decision - run this behavior against several specified inputs rather than a single happy path.

```
040  @pytest.mark.parametrize("problem", ["missing","invalid","unexpected"])
```

decision - group this operation behind a reusable function.

```
041  def test_bad_answers_rejected(course,problem):
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
042      answers={q.question_id:q.correct_choice for q in course.questions if
q.purpose=="diagnostic"}
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
043      key=next(iter(answers))
044      if problem=="missing":
045          answers.pop(key)
046      elif problem=="invalid":
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
047          answers[key]="not-a-choice"
048      else:
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
049          answers["not-a-question"]="A"
050      with pytest.raises(ValueError):
051          score_diagnostic(course,answers)
```

decision - run this behavior against several specified inputs rather than a single happy path.

```
054 @pytest.mark.parametrize("problem", ["duplicate_id","wrong_role","missing_source","invalid_key","
nan"])
```

decision - group this operation behind a reusable function.

```
055  def test_bad_course_rejected(problem):
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
056      data=json.loads((DATA/"course.json").read_text())
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
057      q=next(q for q in data["questions"] if q["purpose"]=="diagnostic")
058      if problem=="duplicate_id":
059          data["questions"].append(copy.deepcopy(q))
060      elif problem=="wrong_role":
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
061          q["purpose"]="practice"
062      elif problem=="missing_source":
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
063          q["source_id"]="missing"
064      elif problem=="invalid_key":
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
065          q["correct_choice"]="not-a-choice"
066      else:
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
067          data["topics"][0]["importance"]=float("nan")
068      with pytest.raises(ValueError):
069          validate_course(data)
```

decision - group this operation behind a reusable function.

```
072  def test_export_roundtrip_and_actual_weights(course):
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
073      request=PlanRequest(budget_minutes=95,proficiency={"T1":1.0,"T2":0.0,"T3":0.5},requested_sol
ver="exact")
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
074      snapshot=make_snapshot(course,request)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
075      assert snapshot.request==request
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
076      assert snapshot.topic_weights==pytest.approx({"T1":5/6,"T2":2/3,"T3":5/6})
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
077      assert validate_snapshot(json.loads(snapshot.model_dump_json()))==snapshot
```

decision - run this behavior against several specified inputs rather than a single happy path.

```
080  @pytest.mark.parametrize("topic,score", [(t,s) for t in ("T1","T2","T3") for s in (0.0,0.5)])
```

decision - record the unavailable ranking checks as skipped, never passed.

```
081  @pytest.mark.skipif(not callable(getattr(rank,"select_questions",None)), reason="blocked - production ranking & reviewed top-id oracle missing")
```

decision - group this operation behind a reusable function.

```
082  def test_retrieval_frozen_cases(course,topic,score):
083      # require reviewed cases rather than invent expected top ids
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
084      path=DATA/"retrieval_cases.json"
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
085      assert path.read_text().strip(), "reviewed retrieval cases missing"
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
086      cases=json.loads(path.read_text())["cases"]
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
087      case=next(c for c in cases if c["topic_id"]==topic and c["proficiency"]==score)
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
088      from src.contracts import PlanResult
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
089      plan=PlanResult(allocations={t:30 if t==topic else 0 for t in ("T1","T2","T3")},used_minutes=30,unused_minutes=0,utility=0.0,feasible=True,actual_solver="exact",execution_mode="live",elapsed_ms=0.0)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
090      items=rank.select_questions(course,plan,dict.fromkeys(("T1","T2","T3"),score))
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
091      assert items and items[0].question_id in case["acceptable_top_ids"]
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
092      approved={q.question_id for q in course.questions if q.purpose=="practice" and q.topic_id==topic and q.review_status=="approved"}
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
093      assert len(items)<=2
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
094      assert len({i.question_id for i in items})==len(items)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
095      assert all(i.question_id in approved for i in items)
```

code appendix
check_quantum_model.py

actual source - evidence/risi/check_quantum_model.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
001  from math import isclose
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
003  import pytest
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
005  from src.contracts import PlanRequest
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
006  from src.optimizer.model import (
007      binary_energy, build_binary_model, polynomial_energy, binary_to_ising,
008      ising_energy, decode_bits, qiskit_bitstring_to_bits,
009  )
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
011  weights = dict.fromkeys(("T1", "T2", "T3"), 1.0)
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
012  scores = {"T1": 0.8, "T2": 0.2, "T3": 0.4}
```

decision - group this operation behind a reusable function.

```
015  def request(budget):
```

decision - return the computed value or assembled result to the caller.

```
016      return PlanRequest(budget_minutes=budget, proficiency=scores, requested_solver="qaoa")
019  # expected vals from hand calc, p=3.4
```

decision - run this behavior against several specified inputs rather than a single happy path.

```
020  @pytest.mark.parametrize("budget,bits,expected,feasible", [
021      (0, [0]*9, 0.0, True),
022      (90, [0,0,0,0,0,0,1,1,0], 0.0, True),
023      (180, [0,0,0,0,0,0,0,1,1], 0.0, True),
024      (90, [0,0,1,1,1,0,0,0,0], -1.8, True),
025      (180, [1,1,1,1,1,1,0,0,0], -2.4, True),
026      (90, [0,1,1,0,1,0,0,0,0], 1.9, False),
027      (0, [0,1,0,0,0,0,0,0,0], 6.7, False),
028      (180, [0,1,0,0,0,0,1,0,1], 3.3, False),
029      (90, [0]*9, 30.6, False),
```

```
030      (0, [1,0,0,0,0,0,0,0,0], 3.2, False),
031  ])
```

decision - group this operation behind a reusable function.

```
032  def test_known_energies_and_decoding(budget, bits, expected, feasible):
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
033      profile = request(budget)
```

decision - expand the profile-specific cost, never a hard-coded student cost.

```
034      binary = build_binary_model(weights, profile)
```

decision - convert those coefficients into z & zz form.

```
035      ising = binary_to_ising(binary)
036      for energy in (binary_energy(bits, weights, profile),
037                     polynomial_energy(bits, binary), ising_energy(bits, ising)):
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
038          assert isclose(energy, expected, rel_tol=0, abs_tol=1e-8)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
039      assert decode_bits(bits, weights, profile)["feasible"] is feasible
```

decision - group this operation behind a reusable function.

```
042  def test_decode_reference_and_partial_budget():
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
043      bits = [0,0,1,1,1,0,0,0,0]
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
044      decoded = decode_bits(bits, weights, request(95))
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
045      assert decoded["allocations"] == {"T1": 0, "T2": 60, "T3": 30}
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
046      assert decoded["used_minutes"] == 90
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
047      assert decoded["unused_minutes"] == 5
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
048      assert decoded["feasible"]
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
049      assert isclose(decoded["utility"], 1.8)
```

decision - group this operation behind a reusable function.

```
052  def test_qiskit_display_order():
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
053      assert qiskit_bitstring_to_bits("000011100") == [0,0,1,1,1,0,0,0,0]
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
054      assert qiskit_bitstring_to_bits("100000000") == [0,0,0,0,0,0,0,0,1]
```

decision - run this behavior against several specified inputs rather than a single happy path.

```
@pytest.mark.parametrize("bits", [[0]*8, [0]*10, [0]*8+[2], [False]+[0]*8, [0.0]*9])
```

decision - group this operation behind a reusable function.

```
def test_rejects_malformed_bits(bits):
    with pytest.raises(ValueError):
        binary_energy(bits, weights, request(90))
    with pytest.raises(ValueError):
        decode_bits(bits, weights, request(90))
```

decision - group this operation behind a reusable function.

```
def test_zero_utility_and_nonunit_weights():
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    profile = PlanRequest(budget_minutes=90, proficiency={"T1": 1.0, "T2": 1.0, "T3": 1.0},
requested_solver="qaoa")
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    weights = {"T1": 0.5, "T2": 0.75, "T3": 1.0}
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    bits = [0,0,0,0,0,0,1,1,0]
```

decision - expand the profile-specific cost, never a hard-coded student cost.

```
    binary = build_binary_model(weights, profile)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
    assert binary_energy(bits, weights, profile) == 0
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
    assert isclose(ising_energy(bits, binary_to_ising(binary)), 0, abs_tol=1e-8)
```

decision - group this operation behind a reusable function.

```
def test_nonunit_weights_energy():
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    weights = {"T1": 0.5, "T2": 0.75, "T3": 1.0}
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    bits = [0,0,1,1,1,0,0,0,0]
    # vals .1, .6, .6; util = 1.5
```

decision - expand the profile-specific cost, never a hard-coded student cost.

```
    binary = build_binary_model(weights, request(90))
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
    assert isclose(binary_energy(bits, weights, request(90)), -1.5)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
    assert isclose(polynomial_energy(bits, binary), -1.5, abs_tol=1e-8)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
    assert isclose(ising_energy(bits, binary_to_ising(binary)), -1.5, abs_tol=1e-8)
```

code appendix check_qaoa.py

actual source - evidence/risi/check_qaoa.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

```
001  """r3 self-checks"""
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
002  from math import isclose
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
004  import pytest
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
005  from qiskit.quantum_info import Statevector
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
007  from src.contracts import PlanRequest
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
008  from src.optimizer.model import build_binary_model, binary_to_ising, binary_energy
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
009  from src.optimizer.quantum_demo import cost_operator, select_sample
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
011  weights = {"T1": 1.0, "T2": 1.0, "T3": 1.0}
```

decision - run this behavior against several specified inputs rather than a single happy path.

```
014  @pytest.mark.parametrize("budget,bits", [
015      (0, [0]*9),
016      (90, [0,0,1,1,1,0,0,0,0]),
017      (180, [1,1,1,1,1,1,0,0,0]),
018      (90, [0,1,1,0,1,0,0,0,0]),
019  ])
```

decision - group this operation behind a reusable function.

```
020  def test_operator_energy(budget, bits):
```

decision - build the shared validated planning input.

```
021      request = PlanRequest(budget_minutes=budget, proficiency={"T1": .8, "T2": .2, "T3": .4},
requested_solver="qaoa")
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
022      model = binary_to_ising(build_binary_model(weights, request))
```

decision - package the converted cost for qiskit while retaining the constant.

```
023      op, offset = cost_operator(model)
```

decision - simulate amplitudes of the bound circuit on a regular computer.

```
024      state = Statevector.from_label("".join(str(bit) for bit in reversed(bits)))
```

decision - calculate real expected energy & add the saved constant offset.

```
025      energy = float(state.expectation_value(op).real + offset)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
026      assert isclose(energy, binary_energy(bits, weights, request), abs_tol=1e-8)
```

decision - group this operation behind a reusable function.

```
029  def test_sample_ties_and_feasibility():
```

decision - build the shared validated planning input.

```
030      request = PlanRequest(budget_minutes=30, proficiency=dict.fromkeys(weights, 0.0),
requested_solver="qaoa")
031      # two valid equal-utility plans, second block without first is invalid
```

decision - draw 1024 measurements & normalize count values for json.

```
032      counts = {"000000100": 2, "000000001": 1, "000000010": 7}
```

decision - filter final samples & choose the best feasible one actually measured.

```
033      best, bits, rate = select_sample(counts, weights, request)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
034      assert best["allocations"] == {"T1": 30, "T2": 0, "T3": 0}
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
035      assert bits == [1,0,0,0,0,0,0,0,0]
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
036      assert rate == .3
```

decision - group this operation behind a reusable function.

```
039  def test_no_feasible_sample():
```

decision - build the shared validated planning input.

```
040      request = PlanRequest(budget_minutes=90, proficiency=dict.fromkeys(weights, 0.0),
requested_solver="qaoa")
```

decision - filter final samples & choose the best feasible one actually measured.

```
041      best, bits, rate = select_sample({"000000000": 1024}, weights, request)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
042      assert best is None
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
043      assert bits is None
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
044      assert rate == 0
```

decision - group this operation behind a reusable function.

```
047  def test_fewer_blocks_tie():
```

decision - build the shared validated planning input.

```
048      request = PlanRequest(budget_minutes=30, proficiency=dict.fromkeys(weights, 1.0),
requested_solver="qaoa")
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
049      best, _, _ = select_sample({"000000001": 3, "001000000": 1}, weights, request)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
050      assert best["used_minutes"] == 0
```

decision - group this operation behind a reusable function.

```
054  def test_saved_run():
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
055      import json
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
056      from pathlib import Path
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
057      from src.intelligence.validate import validate_snapshot
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
058      from src.optimizer.classical import solve_plan
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
059      from src.optimizer.quantum_demo import load_profile
```

decision - locate the repository from this script rather than assuming the caller's directory.

```
061      root = Path(__file__).resolve().parents[2]
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
062      report = json.loads((root / "evidence/risi/qaoa_run.json").read_text())
```

decision - revalidate saved inputs before starting calculations.

```
063      snapshot = validate_snapshot(report["profile"])
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
064      quantum = report["quantum"]
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
065      assert report["execution_mode"] == "live"
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
066      assert quantum["qubits"] == 9
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
067      assert quantum["p"] == 1
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
068      assert quantum["seed"] == 7
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
069      assert quantum["shots"] == sum(quantum["counts"].values()) == 1024
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
070      assert 0 < quantum["objective_evaluations"] == len(quantum["trace"]) <= 30
```

decision - compute the exact baseline for the identical profile.

```
071      exact = solve_plan(snapshot.topic_weights, snapshot.request.model_copy(update={"requested_so
lver": "exact"}))
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
072      assert exact.allocations == report["exact"]["allocations"]
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
073      assert isclose(exact.utility, report["exact"]["utility"])
```

decision - filter final samples & choose the best feasible one actually measured.

```
074      best, bits, rate = select_sample(quantum["counts"], snapshot.topic_weights,
snapshot.request)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
075      assert best is not None
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
076      assert best == quantum["best_sample"]
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
077      assert bits == quantum["best_bits"]
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
078      assert rate == quantum["feasible_shot_rate"]
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
079      assert isclose(report["absolute_utility_gap"], abs(exact.utility - best["utility"]))
```

code appendix check_r4.py

actual source - evidence/risi/check_r4.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
001  import copy
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
002  import json
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
003  from pathlib import Path
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
005  import pytest
```

decision - bring in the named tools; see the file map & import explanations in code_notes.md.

```
007  from evidence.risi.replay_comparison import audit_report
```

decision - group this operation behind a reusable function.

```
010  def report():
```

decision - return the computed value or assembled result to the caller.

```
011      return json.loads(Path(__file__).with_name("r4_comparison.json").read_text())
```

decision - group this operation behind a reusable function.

```
014  def test_clean_committed_run():
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
015      saved=report()
```

decision - check saved evidence before displaying it as a replay.

```
016      snapshot=audit_report(saved)
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
017      assert snapshot.model_dump(mode="json") == json.loads(Path(__file__).with_name("fictional_pr
ofile.json").read_text())
```

decision - check this actual result against the expected contract, hand calculation or saved evidence.

```
018      assert saved["quantum"]["best_sample"]["feasible"]
```

decision - run this behavior against several specified inputs rather than a single happy path.

```
021  @pytest.mark.parametrize("field,val", [("qubits",8),("shots",100),("seed",9),("feasible_shot_rat
e",1.0),("raw_best_utility",99.0)])
```

decision - group this operation behind a reusable function.

```
022  def test_changed_result_rejected(field,val):
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
023      saved=copy.deepcopy(report())
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
024      saved["quantum"][field]=val
```

```
    with pytest.raises(ValueError):
        audit_report(saved)
```

decision - group this operation behind a reusable function.

```
def test_changed_profile_rejected():
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    saved=report()
```

decision - store this intermediate value so later steps can reuse it without changing the input.

```
    saved["profile"]["request"]["proficiency"]["T2"]=1.0
    with pytest.raises(ValueError):
        audit_report(saved)
```

practice - trace it yourself

trace a valid sample

use bits [0,0,1,1,1,0,0,0,0] with the unit-weight oracle. identify topic minutes, count selected blocks, decode slack, calculate utility & energy. answer - 0/60/30, 3 selected, 0 slack, utility 1.8, energy -1.8.

trace an invalid sample

use [0,1,1,0,1,0,0,0,0]. the budget equality holds, but T1 selects its second block without its first. utility is .1+.8+.6 = 1.5. P=3.4, so energy is -1.5+3.4=1.9. the decoder must reject it even though the displayed total is 90 minutes.

find the common mistakes

using test weights instead of exported weights changes the problem.

forgetting the constant offset breaks numeric energy comparisons.

reading count strings left-to-right as qubit indices decodes the wrong plan.

counting distinct strings instead of shot frequencies gives the wrong feasible rate.

treating optimizer_success=false as zero feasible samples confuses convergence with feasibility.

reporting the exact optimum as a quantum answer hides an unsuccessful experiment.

calling skipped retrieval tests passed hides an incomplete integration.

saying you rehearsed or reviewed code when only an assistant did it misstates evidence.

you are ready to rehearse when

you can explain coefficient, slack, phase, expectation, shot & feasibility using one example each; trace both samples above; run the saved replay; & state exactly what is incomplete. if one step feels vague, return to its numbered lesson rather than memorizing a sentence.